

Chapter1: Introduction

United International College

Competitive programming



The International Collegiate Programming Contest is an algorithmic programming contest for college students. Teams of three, representing their university, work to solve the most real-world problems, fostering collaboration, creativity, innovation, and the ability to perform under pressure. Through training and competition, teams challenge each other to raise the bar on the possible. Quite simply, it is the oldest, largest, and most prestigious programming contest in the world.

Goal

Given a problem, we want to

- solve it efficiently
- by using algorithms and data structures,
- convert our solution into a program,
- do it as quickly as possible (under pressure)
- and do it correctly (without bugs)

> This course will exercise this process

For this course

- Course mode
 - Lecture + in-class exercise + discussion
- The course will require knowledge about:
 - Programming language, data structure, algorithm

For this course

- If you have already learn algorithm and data structure
 - Take it as a review.
 - Think more about how to apply algorithm to solve problem.
 - And practice more.
- If you have not learn algorithm and data structure
 - You may need to review after class to better understand the algorithm mentioned.
- If you want to participate the competition
 - You will need to practice every week to get familiar with the problem set for the contest.

For this course

- This course will emphasise on
 - **Practice**
 - Application of problem solving strategy and algorithm
 - Speed
 - **Logical thinking**
 - **Discussion**
 - Team work

How

- Study common types of problems
- Show common applications of algorithms and data structures
- Introduce other common algorithms and data structures
- Go over some commonly used theory
- Practice problem solving
- Practice programming
- More practice
- More practice

Outline

1. Introduction	Lec 1		
a. Competitive Programming			
b. Tips to be Competitive			
2. Data Structures and Libraries	Lec 1-2	5. Mathematics	Lec 10
a. Linear DS with Built-in Libraries		a. Ad Hoc Mathematics Problems	
b. Non-Linear DS with Built-in Libraries		b. Number Theory	
c. Other Data Structures			
3. Problem Solving Paradigms	Lec 3-6	6. String Processing	Lec 11
a. Complete Search		a. Basic String Processing Skills	
b. Divide and Conquer		b. Ad Hoc String Processing Problems	
c. Greedy		c. String Matching	
d. Dynamic Programming			
4. Graph	Lec 7-9	7. (Computational) Geometry	Lec 12
a. Graph Traversal		a. Basic Geometry Objects with Libraries	
b. Minimum Spanning Tree		b. Algorithm on Polygon with Libraries	
c. Shortest Paths			
d. Network Flow		8. Other Advanced Topics	Lec 13

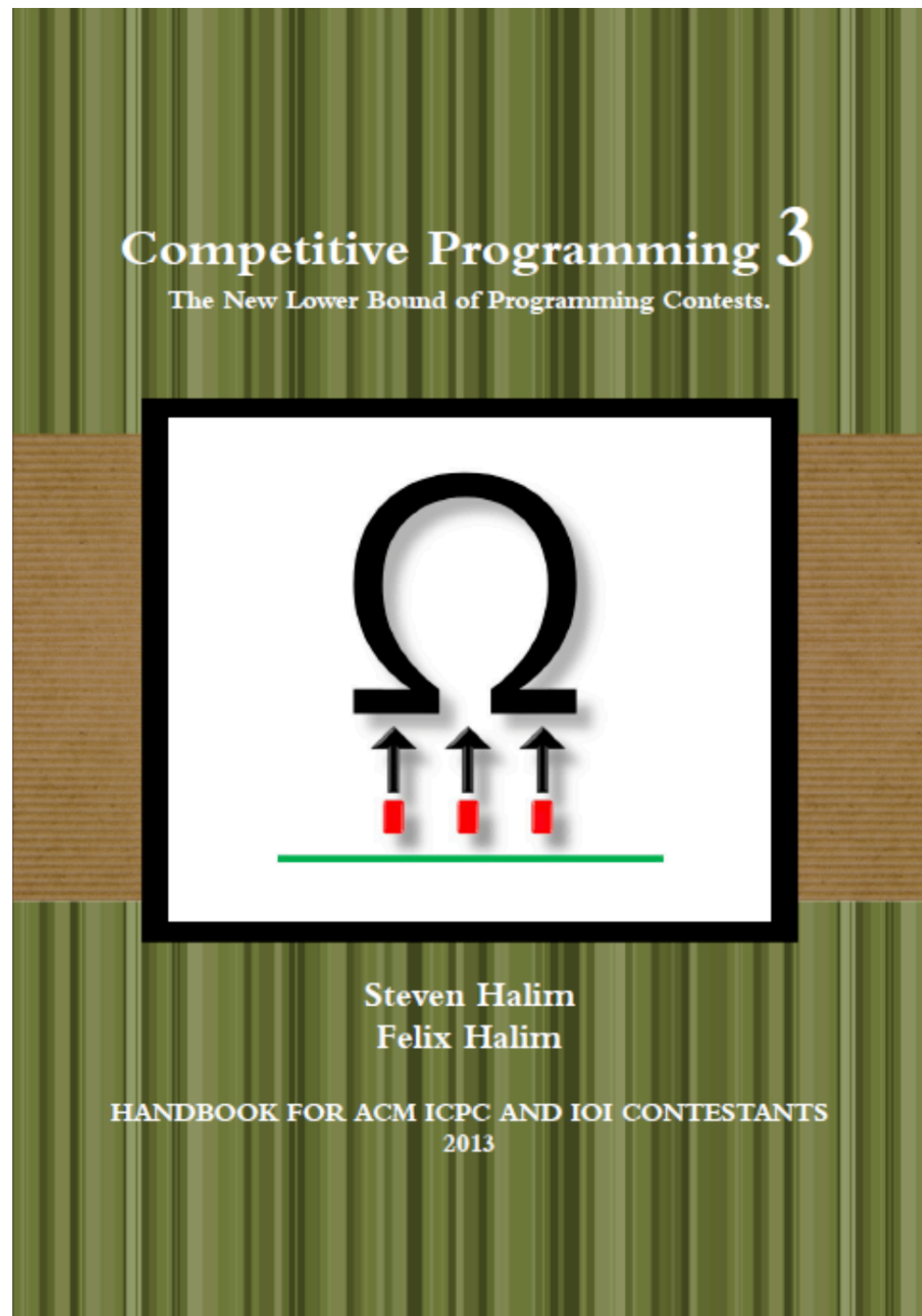
Assessment methods

Type of Assessment Methods	Weighting	CILOs to be addressed	Description of Assessment Tasks
P r o g r a m m i n g Exercise	15%	1	Exercise will give students the hand on experience to solve some simple programming task.
P r o g r a m m i n g Assignment	30%	1,2,3	Programming assignments measure the students' ability to solve practical problems.
Quizzes	25%	1,2	The quizzes assess the major learning outcomes achieved by the students till half of the semester.
Final Contest	30%	2,3	This final contest aims to assess the major learning outcomes achieved by students upon completion of the course.

Assessment methods

- **Programming exercise** : The instructor arranges tutorials in labs where students can discuss algorithm design and practice programming skills.
- **Programming Assignment**: Students are required to implement algorithms to solve problems. At the first 1-2 assignments, students are expected to finish the assignment independently, and then every three students can form a team to finish the following 1-2 assignments.
- **Quiz (Concept; Algorithm design; Programming)**: The quizzes assess the major learning outcomes achieved by the students till half of the semester.
- **Final contest (group)**: Students work in teams and cooperate with each other to design solutions and programming in the final contest, and to experience the competitive programming.

Reference



Book:

Steven Halim and Felix Halim:

[Competitive Programming 3rd](https://github.com/prasadgujar/CompetitiveProgramming/blob/master/book/Competitive%20Programming%203.pdf)

<https://github.com/prasadgujar/CompetitiveProgramming/blob/master/book/Competitive%20Programming%203.pdf>

Important reference:

CS3233 - Competitive Programming, Steven Halim, SoC, NUS

A competitive programming course
-Reykjavik University -Bjarki Ágúst Guðmundsson

Algorithm Design -Princeton - Jon Kleinberg and Éva Tardos

Further reading

Books

- Antti Laaksonen: [*Guide to Competitive Programming: Learning and Improving Algorithms Through Contests*](#)
 - PDF of the book is available from [Springer Link](#) from Purdue IP addresses.
 - A free earlier version of the book titled "Competitive Programmer's Handbook" [Author's Site](#)
- Johan Sannemo: [*Principles of Algorithmic Problem Solving*](#)
- Steven S Skiena and Miguel A. Revilla: [*Programming Challenges: The Programming Contest Training Manual*](#)
 - PDF of the book is available from [Springer Link](#) from Purdue IP addresses.
- Steven Halim and Felix Halim: [*Competitive Programming*](#)
 - Problems on UVa and Kattis associated with the book: [Methods to Solve](#)

Online course:

<http://web.stanford.edu/class/cs97si/>

<https://www.coursera.org/learn/competitive-programming-core-skills/resources/MyBUH>

Useful links

<https://leetcode.com/problemset/all/>

<https://cs.stanford.edu/group/acm/home.html>

<http://stanford.edu/~liszt90/acm/notebook.html>

<http://poj.org/> <https://onlinejudge.org/>

<http://train.usaco.org/usacogate/>



Weekly Contest

- <https://codeforces.com>

Schedule

Ch1. Introduction

- Competitive Programming
- Tips to be Competitive
- Problem examples

Ch2. Data Structures and Libraries

- Why we need data structures
- Data Structures
- Sorting

Ch3. Problem Solving Paradigms

- Complete Search
- Divide and Conquer

Ch4 Greedy

Ch5-6 Dynamic Programming

Schedule

Ch7. Graph

- Graph Traversal

- Minimum Spanning Tree

- Shortest path

Ch8. Network Flow

Ch9. Graph matching

Ch10. Mathematics

- Ad Hoc Mathematics Problems

- Number Theory

Ch11 String Processing

- Basic String Processing Skills

- Ad Hoc String Processing Problems

- String Matching

Ch12. (Computational) Geometry

- Basic Geometry Objects with Libraries

- Algorithm on Polygon with Libraries

Ch13. Other Advanced Topics

Schedule

Ch1. Introduction

Ch2. Data Structures and Libraries

Ch3. Problem Solving Paradigms

Ch4 Greedy

Ch5 Dynamic Programming

Ch6 Dynamic Programming - cont

Ch7. Graph - shortest path

Ch8. Network Flow

Ch9. Graph matching

Ch10. Mathematics

3
Individual
assignment

5 Group
exercise

Random
quiz

Individual

Mid-
Quiz

Randomly
grouping

Reading week

Random
quiz

5
Group
exercise

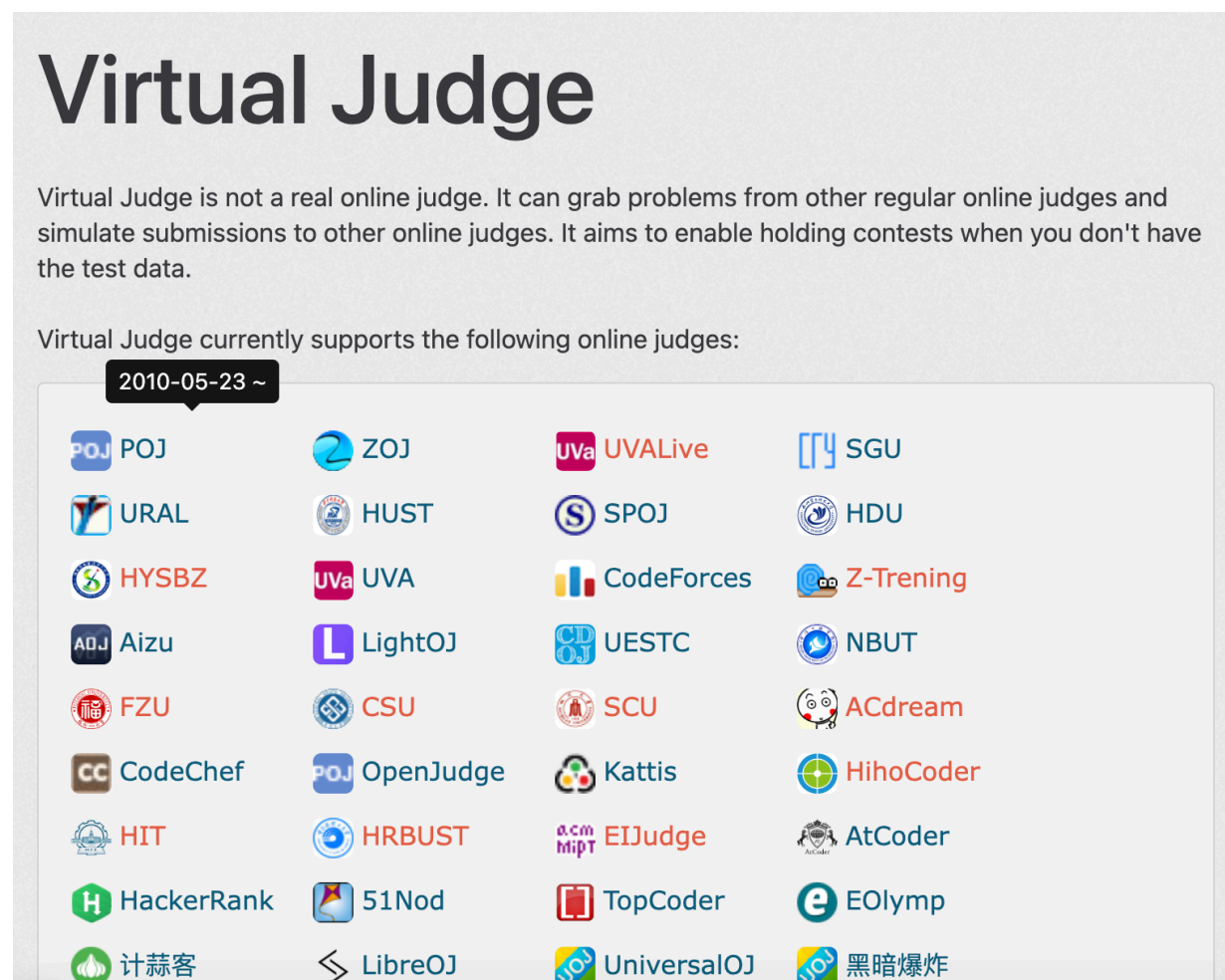
Random
quiz

Voluntary
grouping

Final contest

Exercise

- We will post exercise during class time.
- Firstly you need to discuss in group and have idea on the problem, submit pseudo code (50%)
- Then We will put the problem in Virtual Judge (<https://vjudge.net/>). (50%)
- Copying code or any later submission will get 0.



LET'S START OUR JOURNEY

Example problem

Problem description

Write a program that multiplies pairs of integers.

Input description

Input starts with one line containing an integer T , where $1 \leq T \leq 100$, denoting the number of test cases. Then T lines follow, each containing a test case. Each test case consists of two integers A, B , where $-2^{20} \leq A, B \leq 2^{20}$, separated by a single space.

Output description

For each test case, output one line containing the value of $A \times B$.

Sample input	Sample output
4	12
3 4	0
13 0	8
1 8	10000
100 100	

Example Solution

```
#include <iostream>
using namespace std;

int main() {
    int T;
    cin >> T;

    for (int t = 0; t < T; t++) {

        int A, B;
        cin >> A >> B;

        cout << A * B << endl;
    }

    return 0;
}
```

Example Solution

```
#include <iostream>
using namespace std;

int main() {
    int T;
    cin >> T;

    for (int t = 0; t < T; t++) {

        int A, B;
        cin >> A >> B;

        cout << A * B << endl;
    }

    return 0;
}
```

Is this solution correct?

What if $A=B=2^{20}$? The output is 0...

Example Solution

When $A = B = 2^{20}$, the answer should be 2^{40}

> Too big to fit in a 32-bit integer, so it overflows

> Using 64-bit integers should be enough

```
#include <iostream>
using namespace std;

int main() {
    int T;
    cin >> T;

    for (int t = 0; t < T; t++) {

        long long A, B;
        cin >> A >> B;

        cout << A * B << endl;
    }

    return 0;
}
```

Tips to be competitive

Tip 1: Type Fast & Correct • No kidding, this can be important!

- Let's try

- <http://www.typingtest.com>

- Mine: ~42-45 wpm

- Zhao: ~56-65 wpm

- Steven Halim : ~85-90 wpm

- Felix's: ~55-65 wpm

- Familiarize yourself with the positions of the following keyboard keys:

- (,),{,},[,],<,>,'",&|,!,etc

Tips to be competitive

Tip 2: Identify Problem Types

Rate of appearance of different problem types in recent ICPC Asia Regional problem sets (which usually consists of 7-11 problems):

Category	Sub-Category	Frequency
Ad Hoc	Straightforward	1-2
Ad Hoc	Simulation	0-1
Complete Search	Iterative	0-1
Complete Search	Backtracking	0-1
Divide and Conquer		0-1
Greedy	Classic	0
Greedy	Original	1
Dynamic Programming	Classic	0
Dynamic Programming	Original	1-3
Graph		1-2
Mathematics		1-2
StringProcessing		1
Computational Geometry		1
Some Harder Ones		0-1

Tips to be competitive

Quick Test – Identification

- What is the type of this problem?
 - And how many minutes that you think you will need to solve this problem?
- Given an $M \times N$ integer matrix Q ($1 \leq M, N \leq 50$), check if there exists a sub-matrix of Q of size $A \times B$ ($1 \leq A \leq M, 1 \leq B \leq N$) where $\text{mean}(Q) = 7$?

Tips to be competitive

Tip 3: Do Algorithm Analysis

- When solving a problem, our solution has to be fast enough and can not use too much memory
- We also want our solution to be as simple as possible
- We can use Algorithm Analysis to determine if a solution will run within the time limit
- Rule of thumb: 10^9 operations per second
- We want to sort $n \leq 10^6$ integers, and we have 3 seconds.
 - Can we use a simple $O(n^2)$ bubble sort?
 - What about a more complex $O(n \log n)$ merge sort?
- We want to sort $n \leq 10^3$ integers, and we have 3 seconds.
 - Can we now use the simple $O(n^2)$ bubble sort?
- Always go for the simplest solution that will pass the time limit

Tips to be competitive

Tip 4: Master Programming Languages

- You should master at least one (preferably more) programming languages
 - Reduce the amount of time looking at references
 - Use shortcuts, macros, avoid comments
 - Use libraries whenever possible
- Idea: Once you figure out a solution for a problem, you are able to translate it into a bug-free code, and do it fast!

Tips to be competitive

Tip 5: The Art of Testing Code

- Ultimately, we want “Accepted (AC)”
 - i.e. Our code passes the judge’s secret test data
- However, we may instead be given:
 - Presentation Error (PE)
 - Wrong Answer (WA)
 - Time Limit Exceeded (TLE)
 - Memory Limit Exceeded (MLE)
 - Runtime Error (RTE)

Tips to be competitive

Tip 5: The Art of Testing Code

You want to make sure your solution is correct and runs within the time limit

- Or you already know it's wrong, but don't know why
- Try to break your solution by finding a counter example (an input for which your solution gives incorrect output, or takes too long to compute an answer)
- Try edge cases, large inputs, ...

Tips to be competitive

Tip 6: Practice...

Relevant Online Judges

– MAIN: University of Valladolid (UVa) Online Judge

- <http://uva.onlinejudge.org> (Open with Firefox!)



– MISC: ACM ICPC Live Archive

- <https://icpcarchive.ecs.baylor.edu>



– MISC: TopCoder

- <http://www.topcoder.com>



– MISC: USACO

- <http://train.usaco.org>



Tips to be competitive

Tip 7: Team Work

- Practice coding on a blank paper
 - Submit and print strategy
 - Prepare test data challenges
 - The X-factor
-
- You will experience such team work in group assignment and final team contests

Ad Hoc problems

- The simplest kind of problem
- Just do what the problem description tells you
- Straightforward or a simulation
- Time limit is not an issue
- Sometimes long and misleading problem descriptions
- Sometimes tricky edge cases
- Complex problems can be hard to implement

Problem: Cost Cutting

Description

Company XYZ have been badly hit by recession and is taking a lot of cost cutting measures. Some of these measures include giving up office space, going open source, reducing incentives, cutting on luxuries and issuing pink slips.

They have got three (3) employees working in the accounts department and are going to lay-off two (2) of them. After a series of meetings, they have decided to dislodge the person who gets the most salary and the one who gets the least. This is usually the general trend during crisis like this. You will be given the salaries of these 3 employees working in the accounts department. You have to find out the salary of the person who survives.

Problem: Cost Cutting

Input

The first line of input is an integer T ($T < 20$) that indicates the number of test cases. Each case consists of a line with 3 distinct positive integers. These 3 integers represent the salaries of the three employees. All these integers will be in the range [1000, 10000].

Output

For each case, output the case number followed by the salary of the person who survives.

Sample input	Sample output
3	
1000 2000 3000	Case 1: 2000
3000 2500 1500	Case 2: 2500
1500 1200 1800	Case 3: 1500

Problem: SMS Typing

Description

In this problem we will assume that the key pad of our cell phone is arranged as follows.

	abc	def
ghi	jkl	mno
pqrs	tuv	wxyz
	<SP>	

In the above grid each cell represents one key. Here <SP> means a space. In order to type the letter 'a', we must press that key once, however to type 'b' the same key must be repeatedly pressed twice and for 'c' three times. In the same manner, one key press for 'd', two for 'e' and three for 'f'. This is also applicable for the remaining keys and letters. Note that it takes a single press to type a space.

Problem: SMS Typing

Input

The first line of input will be a positive integer T where T denotes the number of test cases. T lines will then follow each containing only spaces and lower case letters. Each line will contain at least 1 and at most 100 characters.

Output

For every case of input there will be one line of output. It will first contain the case number followed by the number of key presses required to type the message of that case. Look at the sample output for exact formatting.

	abc	def
ghi	jkl	mno
pqrs	tuv	wxyz
	<SP>	

Sample input	Sample output
2 welcome to ulab good luck and have fun	Case #1: 29 Case #2: 41